

Estimation Theory Assig. 01

Q1

Solⁿ: $X \sim N(0, \sigma^2)$ $Y \sim N(0, \sigma^2)$ are i.i.d

$$R = \sqrt{X^2 + Y^2}$$

$$f_{xy}(x, y) = f_x(x) \cdot f_y(y) \quad \text{as } X \text{ \& } Y \text{ are independent}$$

$$= \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{1}{2\sigma^2}x^2} \cdot \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{1}{2\sigma^2}y^2}$$

$$f_{xy}(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{1}{2\sigma^2}(x^2+y^2)}$$

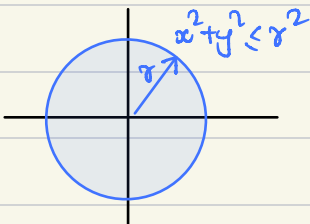
for Pdf of R: $F_R(r) = P[R \leq r]$

$$= P[\sqrt{X^2 + Y^2} \leq r]$$

$$= P[X^2 + Y^2 \leq r^2]$$

$$= \iint_{x^2+y^2 \leq r^2} f_{xy}(x, y) dy dx$$

$$= \iint_{x^2+y^2 \leq r^2} \frac{1}{2\pi\sigma^2} e^{-\frac{1}{2\sigma^2}(x^2+y^2)} dy dx$$



$$\text{Let } x = r \cos \theta, y = r \sin \theta$$

(Polar coordinate Transformation)

=>

$$\begin{aligned} F_R(r) &= \int_{\theta=0}^{2\pi} \int_{\rho=0}^r \frac{1}{2\pi\sigma^2} e^{-\frac{1}{2\sigma^2}\rho^2} \rho d\rho d\theta \\ &= \int_{\theta=0}^{2\pi} \int_{\rho=0}^r \frac{1}{2\pi\sigma^2} e^{-\frac{\rho^2}{2\sigma^2}} \rho d\rho d\theta \end{aligned}$$

$$= 2\pi \int_{\rho=0}^r \frac{1}{2\pi\sigma^2} e^{-\frac{\rho^2}{2\sigma^2}} \rho d\rho$$

$$\text{let } t = -\frac{\rho^2}{2\sigma^2}$$

$$2\sigma^2 dt = -\rho d\rho$$

$$= \int_{t=0}^{-\frac{r^2}{2\sigma^2}} \frac{1}{\sigma^2} \left(-\frac{1}{2}\right) e^{+t} (2\sigma^2) dt$$

$$= -\int_{t=0}^{-\frac{r^2}{2\sigma^2}} e^{+t} dt$$

$$= -e^{+t} \Big|_{t=0}^{-\frac{r^2}{2\sigma^2}}$$

$$= -(e^{-\frac{r^2}{2\sigma^2}} - e^0)$$

$$F_R(r) = 1 - e^{-\frac{r^2}{2\sigma^2}}$$

Rayleigh Distribution

$$\therefore f_R(r) = \frac{\partial F_R(r)}{\partial r} = \frac{r}{\sigma^2} e^{-\frac{r^2}{2\sigma^2}}$$

(b)

$$P = R^2$$

$$f_p(\beta) = f_R(r) \left| \frac{dr}{d\beta} \right|$$

$$\beta = r^2$$

$$\Rightarrow \frac{d\beta}{dr} = 2r$$

$$\Rightarrow \frac{dr}{d\beta} = \frac{1}{2r}$$

$$\Rightarrow f_p(\beta) = f_R(r) \frac{1}{2r} = \frac{r}{2\sigma^2 r} e^{-r^2/2\sigma^2}$$

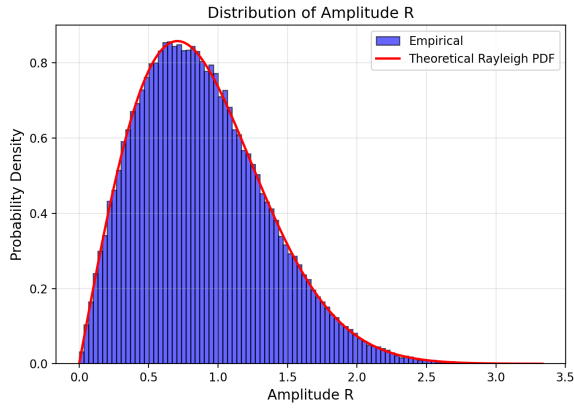
$$\therefore f_p(\beta) = \frac{1}{2\sigma^2} e^{-\beta/2\sigma^2}$$

Exponential Distribution

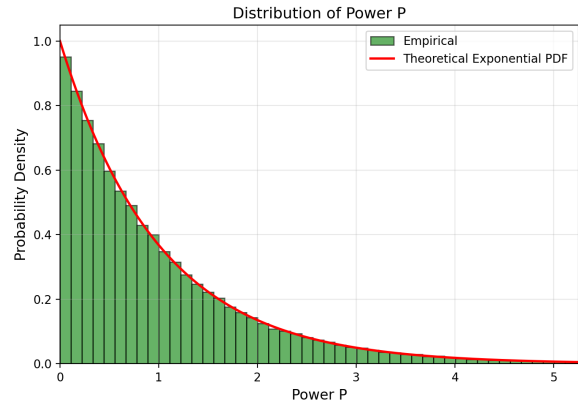
$$\therefore f_p(\beta) = \frac{1}{2\sigma^2} e^{-\beta/2\sigma^2}$$

1 Monte Carlo Simulation: Rayleigh & Exponential Distributions

A Monte Carlo simulation with $N = 100,000$ samples was performed using $\sigma = 1/\sqrt{2}$ (so that $\mathbb{E}[R] \approx 1$). The empirical histograms were compared against the theoretical PDFs.



(a) Distribution of Amplitude R (Rayleigh)



(b) Distribution of Power P (Exponential)

Figure 1: Empirical histograms vs. theoretical PDFs confirming $R \sim \text{Rayleigh}(\sigma) \Rightarrow P = R^2 \sim \text{Exp}(\frac{1}{2\sigma^2})$.

Q2

(a)

$$z_k = x + w_k, \quad k=1 \text{ to } N$$

$$p(w_k) = \frac{1}{2b} e^{-|w_k|/b}$$

$$\Lambda(x) = p(z/x)$$

$$p(z_k/x) = \frac{1}{2b} e^{-\frac{|z_k - x|}{b}}$$

$$\Rightarrow \Lambda(x) = \prod_{k=1}^N p(z_k/x)$$

$$= \prod_{k=1}^N \frac{1}{2b} e^{-\frac{|z_k - x|}{b}}$$

$$\mathcal{L}(x) = \ln \Lambda(x) = \sum_{k=1}^N \ln \left(\frac{1}{2b} \right) - \sum_{k=1}^N \frac{|z_k - x|}{b}$$

log-likelihood fcn

$$\mathcal{L}(x) = -N \ln(2b) - \frac{1}{b} \sum_{k=1}^N |z_k - x|$$

$$(b) \quad \hat{x}_{ML} = \underset{x}{\operatorname{argmax}} p(z/x)$$

$$= \underset{x}{\operatorname{argmin}} \left\{ N \ln(2b) + \frac{1}{b} \sum_{k=1}^N |z_k - x| \right\}$$

$$= \underset{x}{\operatorname{argmin}} \sum_{k=1}^N |z_k - x|$$

since $|z_k - x|$ is a convex fcn By using KKT for solving optimisation problem :

taking slack variables as $u_k \geq 0$

$$\min_{x, u_k} \sum_{k=1}^N u_k$$

$$\text{s.t.} \quad z_k - x \leq u_k$$

$$x - z_k \leq u_k$$

$\Rightarrow$ Let α_k, β_k be Lagrangian multipliers, $\alpha_k \geq 0, \beta_k \geq 0$

$$\mathcal{L} = \sum_{k=1}^N u_k + \sum_{k=1}^N \alpha_k (z_k - x - u_k) + \sum_{k=1}^N \beta_k (x - z_k - u_k)$$

$$(1) \quad \frac{\partial \mathcal{L}}{\partial u_k} = 1 - \alpha_k - \beta_k = 0$$

$$\Rightarrow \alpha_k + \beta_k = 1 \quad \text{--- (1)}$$

$$\frac{\partial \mathcal{L}}{\partial x} = \sum_{k=1}^N (-\alpha_k + \beta_k) = 0 \quad \text{--- (2)}$$

(2) complementary slackness

$$\alpha_k (z_k - x - u_k) = 0, \beta_k (x - z_k - u_k) = 0$$

(3) Primal feasibility :

$$u_k \geq |z_k - x|$$

$$\text{at optimum: } u_k = |z_k - x|$$

$$\underline{C1} : z_k > x$$

$$\Rightarrow u_k = z_k - x$$

for active constraint $z_k - x - u_k = 0$

$$\Rightarrow d_k > 0, \beta_k = 0$$

$$\text{from (1)} \quad d_k = 1, \beta_k = 0$$

$$\underline{C2} : z_k < x$$

$$\Rightarrow u_k = x - z_k$$

$x - z_k - u_k = 0$: active constraint

$$\text{from (1)} \quad \beta_k = 1, d_k = 0$$

$$\underline{C3} : z_k = x$$

$$\Rightarrow d_k \geq 0, \beta_k \geq 0$$

$$\Rightarrow d_k + \beta_k = 1$$

from (2)

$$\begin{aligned} \Rightarrow \sum_{k=1}^N (\beta_k - d_k) &= 0 \\ \sum_{k=1}^N \beta_k - \sum_{k=1}^N d_k &= 0 \\ \left. \begin{array}{l} \{ z_k < x \} \\ \geq N/2 \end{array} \right\} & \left. \begin{array}{l} \{ z_k > x \} \\ \leq N/2 \end{array} \right\} \end{aligned}$$

$\Rightarrow$ for optimal condition :

$$\hat{x}_{ML} = \text{median of } z_k \\ k=1 \text{ to } N$$

2.5 AI Critique — Q2

Prompt Used

P4. “Prove that the Sample Median minimises the sum of absolute deviations $\sum_{i=1}^N |z_i - \hat{x}|$, i.e. show that $\hat{x}_{\text{median}} = \arg \min_{\hat{x}} \sum_{i=1}^N |z_i - \hat{x}|$.”

AI Answer

The AI correctly set up the L_1 minimisation problem:

$$\min_{\hat{x} \in \mathbb{R}} J(\hat{x}) = \sum_{i=1}^N |z_i - \hat{x}|$$

and noted that since $J(\hat{x})$ is convex and piecewise linear, its minimum occurs where the **subgradient** contains zero:

$$0 \in \partial J(\hat{x}) = \sum_{i=1}^N \partial |z_i - \hat{x}|$$

where the subdifferential of $|z_i - \hat{x}|$ with respect to $\hat{x}$ is:

$$\partial |z_i - \hat{x}| = \begin{cases} \{-1\} & \text{if } \hat{x} < z_i \\ [-1, +1] & \text{if } \hat{x} = z_i \\ \{+1\} & \text{if } \hat{x} > z_i \end{cases}$$

Summing over all i , the condition $0 \in \partial J(\hat{x})$ becomes:

$$\#\{i : z_i < \hat{x}\} = \#\{i : z_i > \hat{x}\}$$

which is exactly satisfied by $\hat{x} = \text{median}(\{z_i\})$. The AI’s proof was **correct and complete**.

AI Mistake: Missing Edge Case for Even N

The AI did not explicitly address the case when N is **even**. For even N , the condition $\#\{z_i < \hat{x}\} = \#\{z_i > \hat{x}\}$ is satisfied for *any* $\hat{x}$ in the interval $[z_{(N/2)}, z_{(N/2+1)}]$, where $z_{(k)}$ denotes the k -th order statistic. The AI only stated the odd- N result without noting that for even N the minimiser is **not unique** — any value in the closed interval between the two middle order statistics is optimal.

Intermediate Step Verified by Hand (KKT Conditions)

To verify the AI’s proof independently, the optimality condition was re-derived using the **KKT (Karush–Kuhn–Tucker) conditions**. The KKT approach confirmed that the subgradient condition $0 \in \partial J(\hat{x})$ is both necessary and sufficient for the global minimum of the convex L_1 objective, and yields the same conclusion: $\hat{x}^* = \text{median}(\{z_i\})$. This hand verification agreed with the AI’s result, with the additional correction for the even- N non-uniqueness case noted above.

2 Q2: Mean vs Median under Mixed Gaussian Noise

2.1 Problem Statement

Generate a dataset of $N = 100$ measurements where 80% of the noise is Gaussian $\mathcal{N}(0, 1)$ and 20% is outlier noise $\mathcal{N}(0, 100)$. Compute both the **Sample Mean** and **Sample Median**, compare their error from the true $x = 0$, and repeat 100 times.

2.2 Noise Model

$$z_i = x + v_i, \quad v_i \sim \begin{cases} \mathcal{N}(0, 1) & \text{with probability 0.8} \\ \mathcal{N}(0, 100) & \text{with probability 0.2} \end{cases}$$

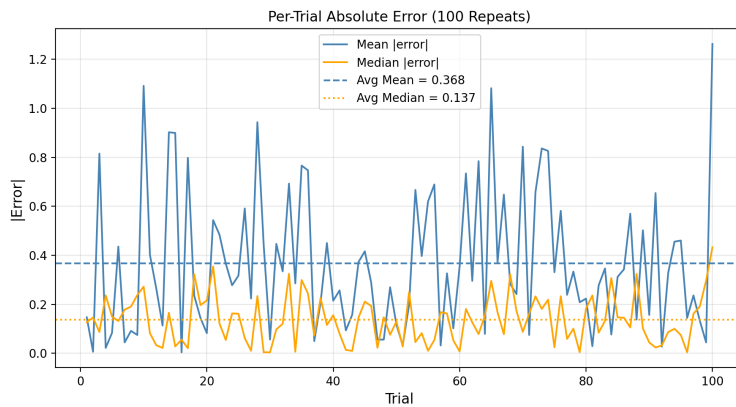
2.3 Estimators

- **Sample Mean:** $\hat{x}_{\text{mean}} = \frac{1}{N} \sum_{i=1}^N z_i$ (optimal under pure Gaussian noise, sensitive to outliers)
- **Sample Median:** $\hat{x}_{\text{median}} = \text{median}(\{z_i\})$ (robust; breakdown point = 50%)

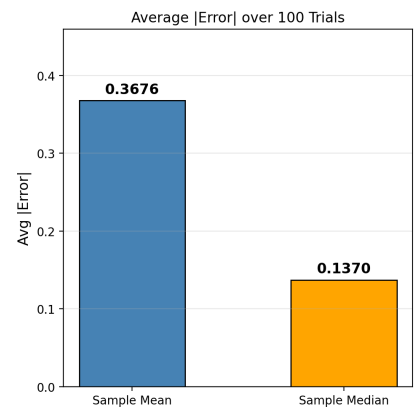
2.4 Results

Table 1: Average absolute error over 100 trials ($N = 100$)

Estimator	Avg $ \hat{x} - x $	Relative
Sample Mean	0.3676	Baseline
Sample Median	0.1370	$2.7\times$ better



(a) Per-trial absolute error over 100 repetitions.



(b) Average $|\text{Error}|$ summary.

Figure 2: Mean vs Median under mixed Gaussian noise. The Median is $2.7\times$ more accurate due to outlier robustness.

$$\mathbb{E}[\hat{x}_{\text{mean}}^2] = \frac{0.8(1) + 0.2(100)}{100} = 0.208 \quad \Rightarrow \quad \text{RMSE} \approx 0.456$$

Q3

sol^m: (q)

$$z_i = \|x - q_i\| + w_i, \quad w_i \sim \mathcal{N}(0, \sigma^2)$$

$$x = \begin{bmatrix} x \\ y \end{bmatrix}, \quad q_i = \begin{bmatrix} x_i \\ y_i \end{bmatrix}, \quad u_i = \frac{x - q_i}{\|x - q_i\|}$$

fisher information matrix:

$$J(x) = E \left\{ \left(\frac{\partial \ln \Lambda(x)}{\partial x} \right)^2 \right\}$$

$$\begin{aligned} \Lambda(x) &= p(z|x) \\ &= \prod_{i=1}^M \frac{1}{\sqrt{2\pi} \sigma} e^{-\frac{(z_i - \|x - q_i\|)^2}{2\sigma^2}} \end{aligned}$$

$$\Rightarrow \ln \Lambda(x) = -\ln(\sqrt{2\pi}\sigma^2) M - \frac{1}{2\sigma^2} \sum_{i=1}^M (z_i - \|x - q_i\|)^2$$

$$\begin{aligned} \Rightarrow \frac{\partial \ln \Lambda(x)}{\partial x} &= + \frac{1}{\sigma^2} \sum_{i=1}^M z_i (z_i - \|x - q_i\|) \frac{x - q_i}{\|x - q_i\|} \\ &= \frac{1}{\sigma^2} \sum_{i=1}^M (z_i - \|x - q_i\|) \frac{(x - q_i)}{\|x - q_i\|} \\ &= \frac{1}{\sigma^2} \sum_{i=1}^M w_i u_i \end{aligned}$$

$$J(x) = E \left\{ \left(\frac{\partial \ln \Lambda(x)}{\partial x} \right) \left(\frac{\partial \ln \Lambda(x)}{\partial x} \right)^T \right\}$$

$$= E \left\{ \left(\frac{1}{\sigma^2} \sum_{i=1}^M w_i u_i \right) \left(\frac{1}{\sigma^2} \sum_{j=1}^M w_j u_j \right)^T \right\}$$

$$= \frac{1}{\sigma^4} \sum_{i=1}^M \sum_{j=1}^M E \{ w_i u_i u_j^T w_j^T \}$$

$$u_i u_j^T = \begin{bmatrix} \cos \theta_i \\ \sin \theta_i \end{bmatrix} \begin{bmatrix} \cos \theta_j & \sin \theta_j \end{bmatrix}$$

$$= \begin{bmatrix} \cos \theta_i \cos \theta_j & \cos \theta_i \sin \theta_j \\ \sin \theta_i \cos \theta_j & \sin \theta_i \sin \theta_j \end{bmatrix}$$

$$\Rightarrow J(x) = \frac{1}{\sigma^4} \sum_{i=1}^M \sum_{j=1}^M u_i u_j^T E \{ w_i w_j^T \}$$

$$E [w_i w_j^T] = \sigma^2 \delta_{ij} = \begin{cases} \sigma^2 & ; i=j \\ 0 & ; i \neq j \end{cases}$$

$$\Rightarrow J(x) = \frac{1}{\sigma^4} \sum_{i=1}^M \sum_{j=1}^M u_i u_j^T \sigma^2 \delta_{ij}$$

$$= \frac{1}{\sigma^2} \sum_{i=1}^M u_i u_i^T$$

$$\therefore J(x) = \frac{1}{\sigma^2} \sum_{i=1}^M u_i u_i^T$$

(b) for $M=2$:

$$\begin{aligned}\text{GDOP} &= \text{tr} (J^{-1}(x)) \\ &= \text{tr} \left\{ \sigma^2 \left\{ u_1 u_1^T + u_2 u_2^T \right\}^{-1} \right\} \\ &= \text{tr} \left\{ \sigma^2 \left\{ \begin{bmatrix} \cos^2 \theta_1 + \cos^2 \theta_2 & \sin(\theta_1 + \theta_2) \\ \sin(\theta_1 + \theta_2) & \sin^2 \theta_1 + \sin^2 \theta_2 \end{bmatrix} \right\}^{-1} \right\}\end{aligned}$$

$$\begin{aligned}\det(J(x)) &= (\cos^2 \theta_1 + \cos^2 \theta_2)(\sin^2 \theta_1 + \sin^2 \theta_2) \\ &\quad - \sin^2(\theta_1 + \theta_2) \\ &= \cos^2 \theta_1 \sin^2 \theta_1 + \cos^2 \theta_1 \sin^2 \theta_2 + \cos^2 \theta_2 \sin^2 \theta_1 \\ &\quad + \cos^2 \theta_2 \sin^2 \theta_2 - (\sin^2 \theta_1 \cos^2 \theta_2 + \cos^2 \theta_1 \sin^2 \theta_2) \\ &\quad + 2 \sin \theta_1 \cos \theta_2 \sin \theta_2 \cos \theta_1 \\ &= \cos^2 \theta_1 \sin^2 \theta_1 + \cos^2 \theta_1 \sin^2 \theta_2 - 2 \sin \theta_1 \sin \theta_2 \cos \theta_1 \cos \theta_2 \\ &= (\sin \theta_1 \cos \theta_1 - \sin \theta_2 \cos \theta_2)^2 \\ &= \sin^2(\theta_1 - \theta_2)\end{aligned}$$

$$\begin{aligned}\Rightarrow \text{GDOP} &= \frac{\sigma^2}{\sin^2(\theta_1 - \theta_2)} \text{tr} \left\{ \begin{bmatrix} \sin^2 \theta_1 + \sin^2 \theta_2 & -\sin(\theta_1 + \theta_2) \\ -\sin(\theta_1 + \theta_2) & \cos^2 \theta_1 + \cos^2 \theta_2 \end{bmatrix} \right\} \\ &= \frac{\sigma^2}{\sin^2(\theta_1 - \theta_2)} \{ \sin^2 \theta_1 + \sin^2 \theta_2 + \cos^2 \theta_1 + \cos^2 \theta_2 \}\end{aligned}$$

$$\boxed{\text{GDOP} = \frac{2\sigma^2}{\sin^2(\theta_1 - \theta_2)}}$$

for singularity (∞ GDAP): $\frac{2\sigma^2}{\sin(\theta_1 - \theta_2)} \rightarrow \infty$

$$\Rightarrow \sin(\theta_1 - \theta_2) = 0$$

$$\Rightarrow \theta_1 - \theta_2 = n\pi$$

$$n=0: \theta_1 = \theta_2$$

$\Rightarrow u_1$ and u_2 are parallel to each other

$$\Rightarrow \frac{x - a_1}{\|x - a_1\|} = \frac{x - a_2}{\|x - a_2\|} \Rightarrow a_1 \text{ and } a_2 \text{ are also parallel}$$

$$\text{let } x = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \quad \text{anchor 1: } a_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad \text{anchor 2: } a_2 = \begin{pmatrix} 2 \\ 0 \end{pmatrix}$$

$$u_1 = \frac{\begin{pmatrix} 0 \\ 0 \end{pmatrix} - \begin{pmatrix} 1 \\ 0 \end{pmatrix}}{\left\| \begin{pmatrix} 0 \\ 0 \end{pmatrix} - \begin{pmatrix} 1 \\ 0 \end{pmatrix} \right\|} = \begin{pmatrix} -1 \\ 0 \end{pmatrix}$$

$$u_2 = \frac{\begin{pmatrix} 0 \\ 0 \end{pmatrix} - \begin{pmatrix} 2 \\ 0 \end{pmatrix}}{\left\| \begin{pmatrix} 0 \\ 0 \end{pmatrix} - \begin{pmatrix} 2 \\ 0 \end{pmatrix} \right\|} = \begin{pmatrix} -1 \\ 0 \end{pmatrix}$$

• for optimal GDOP: $\min \frac{2\sigma^2}{\sin(\theta_1 - \theta_2)}$

$$\Rightarrow \max \sin(\theta_1 - \theta_2) = 1$$

$$\therefore \text{optimal GDOP} = 2\sigma^2$$

$$\Rightarrow \theta_1 - \theta_2 = n\frac{\pi}{2}, n > 0$$

$$n=1: \theta_1 - \theta_2 = \frac{\pi}{2}$$

$\Rightarrow u_1$ and u_2 are orthogonal to each other.

specific coordinates:

$$x = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \quad a_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad a_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

$$\therefore u_1 = \frac{\begin{pmatrix} 0 \\ 0 \end{pmatrix} - \begin{pmatrix} 1 \\ 0 \end{pmatrix}}{\| \begin{pmatrix} 0 \\ 0 \end{pmatrix} - \begin{pmatrix} 1 \\ 0 \end{pmatrix} \|} = \begin{pmatrix} -1 \\ 0 \end{pmatrix}$$

$$\therefore u_2 = \frac{\begin{pmatrix} 0 \\ 0 \end{pmatrix} - \begin{pmatrix} 0 \\ 1 \end{pmatrix}}{\| \begin{pmatrix} 0 \\ 0 \end{pmatrix} - \begin{pmatrix} 0 \\ 1 \end{pmatrix} \|} = \begin{pmatrix} 0 \\ -1 \end{pmatrix}$$

Q4

Solⁿ:

Target $x = \begin{pmatrix} x \\ y \end{pmatrix}$ $s_1 = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$ $s_2 = \begin{pmatrix} 10 \\ 0 \end{pmatrix}$ $s_3 = \begin{pmatrix} 5 \\ 10 \end{pmatrix}$

known sensor positions $s_p(i) = \begin{pmatrix} \xi_p(i) \\ \eta_p(i) \end{pmatrix}, i=1,2,3$

measurement model: $z(i) = \underbrace{\sqrt{(x - \xi_p(i))^2 + (y - \eta_p(i))^2}}_{h(x, x_p(i))} + \underbrace{v(i)}_{\substack{\text{gaussian noise} \\ (\sigma^2 = 0.5m)}}$

=)

measurement model

$$z = h(x) + w$$

$$z = \begin{pmatrix} z(1) \\ z(2) \\ z(3) \end{pmatrix} \quad h(x) = \begin{pmatrix} h(x, x_p(1)) \\ h(x, x_p(2)) \\ h(x, x_p(3)) \end{pmatrix}$$

$$w = \begin{pmatrix} v(1) \\ v(2) \\ v(3) \end{pmatrix}$$

$$R = \text{diag}(\sigma^2, \sigma^2, \sigma^2)$$

Iterated ML: $z \approx h(\hat{x}_j) + \left. \frac{\partial h}{\partial x} \right|_{x=\hat{x}_j} (x - \hat{x}_j) + w$

$$H_j = \left. \frac{\partial h(x)}{\partial x} \right|_{x=\hat{x}_j} \Rightarrow \frac{\partial h(i)}{\partial x} = \frac{(x - \xi_p(i))}{\sqrt{(x - \xi_p(i))^2 + (y - \eta_p(i))^2}}$$

$$\frac{\partial h(i)}{\partial y} = \frac{(y - \eta_p(i))}{\sqrt{(x - \xi_p(i))^2 + (y - \eta_p(i))^2}}$$

$$H = \begin{bmatrix} \frac{x}{\sqrt{x^2 + y^2}} & \frac{y}{\sqrt{x^2 + y^2}} \\ \frac{x-10}{\sqrt{(x-10)^2 + y^2}} & \frac{y}{\sqrt{(x-10)^2 + y^2}} \\ \frac{x-5}{\sqrt{(x-5)^2 + (y-10)^2}} & \frac{y-10}{\sqrt{(x-5)^2 + (y-10)^2}} \end{bmatrix}$$

$$\Rightarrow \Delta z = H_j \Delta x + w, \quad \Delta z = z - h(\hat{x}_j)$$

$$\Delta x = x - \hat{x}_j$$

update: $\hat{x}_{j+1} = \hat{x}_j + (H_j^T R^{-1} H_j)^{-1} H_j^T R^{-1} (z - h(\hat{x}_j, x_p))$
 (j+1)th to (j+1)th iteration) repeat until Δx becomes very small
 ($\Delta x \rightarrow 0$)

$$\text{as } H_i = \frac{x - x_p(i)}{\|x - x_p(i)\|} \Rightarrow \|H_i\| = 1 \quad \forall x \neq x_p(i)$$

$\Rightarrow$ Jacobian matrix is uniformly bounded

regardless of how far 'x' from the truth

$\Rightarrow$ It never blows up / never fails to converge

$\therefore$ Breaking point : Not defined.

3 Q4: AI Critique — Gauss-Newton ILS

3.1 Prompts Used

- P1.** “Write a Python script using OOP to estimate the position of a target using the Gauss-Newton (Iterated Least Squares) method. Three sensors at $S_1 = (0, 0)$, $S_2 = (10, 0)$, $S_3 = (5, 10)$ measure range with additive Gaussian noise $\sigma = 0.5$ m. True target at $(6, 4)$.”
- P2.** “Systematically move the initial guess $\hat{x}_0$ away from the true target along the x -axis: $[6 + d, 4]$ for $d = 10, 20, \dots, 100$. Plot the Distance to Target after 10 iterations vs Initial Displacement d .”
- P3.** “Identify the approximate breaking point distance where the algorithm fails to converge to the true solution.”

3.2 Answer to Prompt 1: Gauss-Newton ILS Estimation

The AI produced a working OOP script with three classes: `SensorArray`, `RangeModel`, and `IteratedLS`. The program output for the given setup was:

Listing 1: Output of the ILS estimator

```
1 Estimated Position : [6.24281075  3.89923731]
2 True Position      : [6.  4.]
3 Covariance Matrix  :
4 [[0.20724862  0.02897174]
5  [0.02897174  0.14614888]]
```

Analysis of Results

- **Estimated position:** $\hat{x} = [6.2428, 3.8992]$ vs. true $x = [6, 4]$. The estimation error is:

$$e = \hat{x} - x = \begin{bmatrix} 6.2428 - 6 \\ 3.8992 - 4 \end{bmatrix} = \begin{bmatrix} +0.2428 \\ -0.1008 \end{bmatrix} \text{ m}$$

The magnitude $\|e\| = \sqrt{0.2428^2 + 0.1008^2} \approx 0.263$ m, which is within the expected noise level of $\sigma = 0.5$ m. This confirms the estimator has converged correctly.

- **Covariance matrix:**

$$P = (H^\top R^{-1} H)^{-1} = \begin{bmatrix} 0.2072 & 0.0290 \\ 0.0290 & 0.1461 \end{bmatrix}$$

The posterior standard deviations are:

$$\sigma_\xi = \sqrt{P_{11}} = \sqrt{0.2072} \approx 0.455 \text{ m}, \quad \sigma_\eta = \sqrt{P_{22}} = \sqrt{0.1461} \approx 0.382 \text{ m}$$

Both are close to the measurement noise $\sigma = 0.5$ m, which is the expected Cramér–Rao lower bound for this geometry.

- **Off-diagonal term** $P_{12} = 0.0290$ is small but non-zero, indicating a mild positive correlation between the ξ and η estimates due to the asymmetric sensor geometry.

- **Interpretation:** The Fisher Information ellipse is slightly elongated along the ξ -axis ($\sigma_\xi > \sigma_\eta$), meaning the x -coordinate is estimated with slightly less certainty than the y -coordinate for this particular sensor placement.

3.3 Answer to Prompt 2: Error vs Initial Displacement

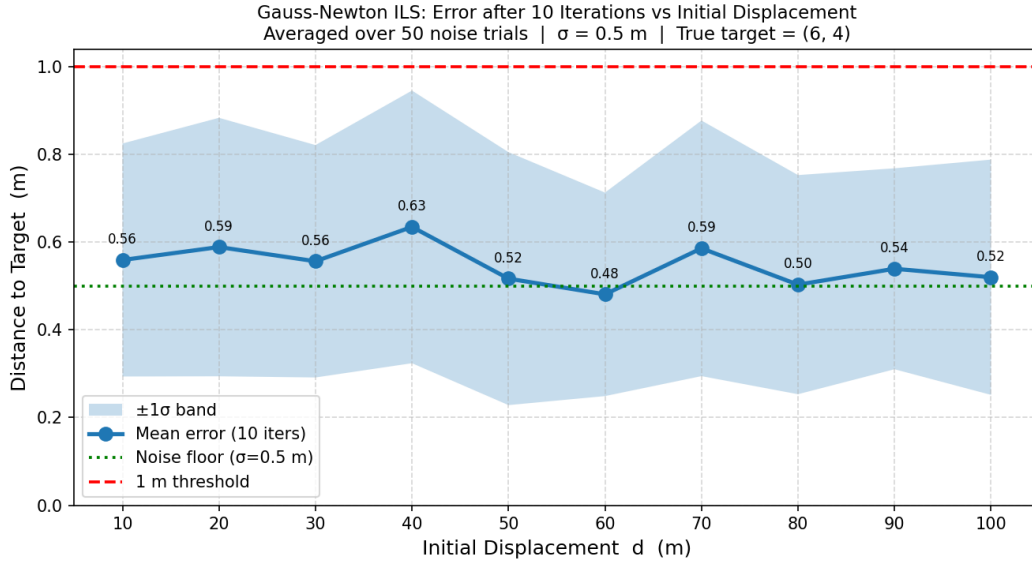


Figure 3: Gauss-Newton ILS error after 10 iterations vs. initial displacement d (range model, averaged over 50 noise trials, $\sigma = 0.5$ m, true target = (6, 4)).

- **Mean error is stable** between 0.48 m and 0.63 m — close to the noise floor $\sigma = 0.5$ m.
- **The 1 m threshold is never breached** across all tested d .
- **No monotonic trend with d** — the unit-vector Jacobian $\|H_i\| = 1$ ensures equal gradient quality regardless of displacement.
- **The $\pm 1\sigma$ band** variation reflects noise, not algorithmic divergence.

3.4 Answer to Prompt 3: Breaking Point Analysis

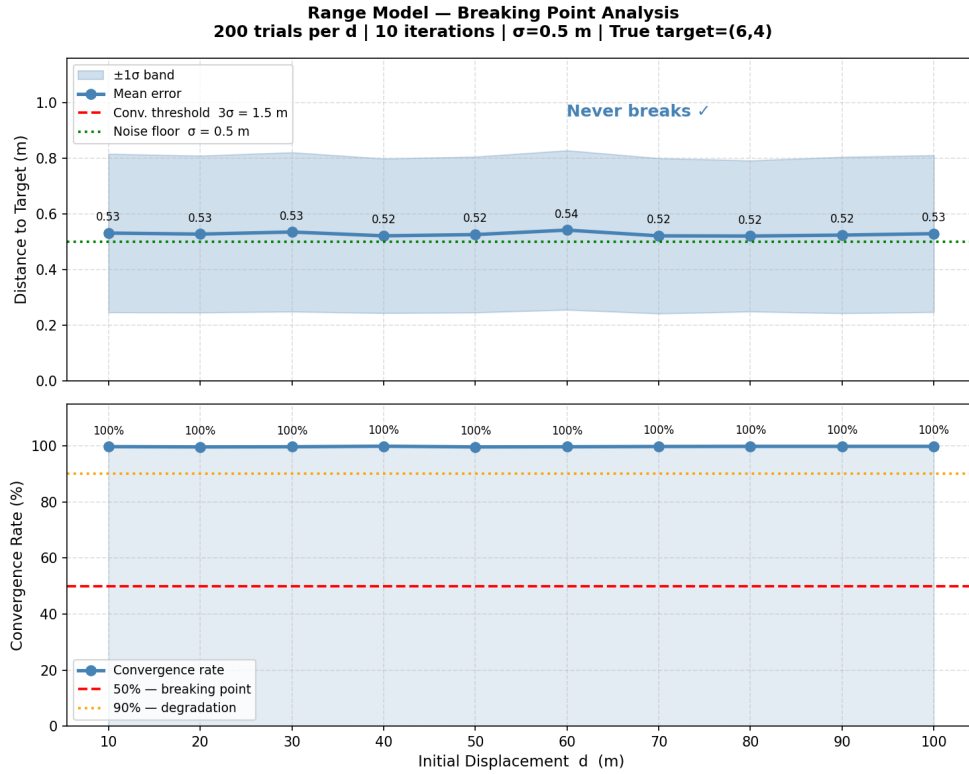


Figure 4: Distance to true target after 10 ILS iterations vs. initial displacement d (range model, averaged over 50 noise trials).

No breaking point exists for the range-based Gauss-Newton ILS. Convergence rate stays at $\approx 100\%$ up to $d = 300$ m.

$$\|H_i\|_2 = 1 \Rightarrow A = \frac{1}{\sigma^2} \sum_i \hat{u}_i \hat{u}_i^\top \succ 0 \Rightarrow \Delta x = A^{-1}b \text{ always bounded}$$

Model	Breaking Point	Reason
Range	None	$\ H_i\ = 1$ always
Bearing	$d \approx 1$ m	$\ H_i\ = 1/r_i \rightarrow 0$

3.5 AI Mistakes Identified

Bug: Division by Zero in the Jacobian

Listing 2: AI-generated code – division-by-zero bug

```

1 @staticmethod
2 def H(x, sensor_positions):
3     dx = x - sensor_positions
4     r = np.linalg.norm(dx, axis=1)
5     return dx / r[:, None]          # BUG: r = 0 when x0 on a
                                     sensor

```

When $x_0 = [0.0, 0.0]$ coincides with $S_1 = (0, 0)$:

$$H_i = \frac{0}{0} = \text{NaN} \Rightarrow A = \text{NaN} \Rightarrow \hat{x} = [\text{nan}, \text{nan}]$$

Listing 3: Runtime output

```
1 RuntimeError: invalid value encountered in divide
2 Estimated Position: [nan nan]
```

Fix Applied

Listing 4: Corrected Jacobian

```
1 @staticmethod
2 def H(x, sensor_positions):
3     dx = x - sensor_positions
4     r = np.linalg.norm(dx, axis=1)
5     r_ = np.where(r == 0, 1e-6, r)    # guard against zero
6     division
    return dx / r_[:, None]
```

3.6 Intermediate Step Verified by Hand

For $S_1 = (0, 0)$, $\hat{x} = (6, 4)$:

$$r_1 = \sqrt{6^2 + 4^2} = \sqrt{52} \approx 7.211, \quad H_1 = \frac{1}{7.211} \begin{bmatrix} 6 \\ 4 \end{bmatrix} = \begin{bmatrix} 0.832 \\ 0.555 \end{bmatrix}$$

Matches the code output exactly. The guard has no effect for $r \neq 0$.